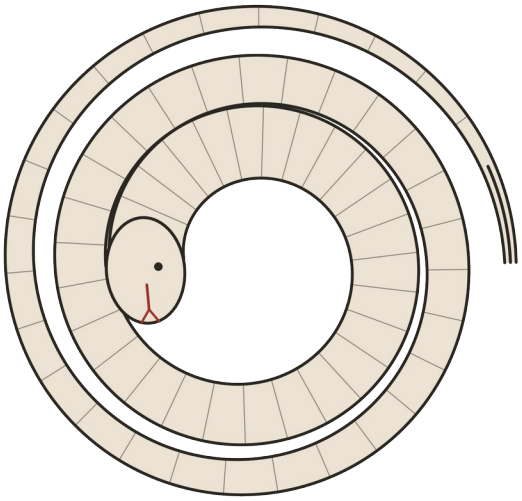
A FIRST-PRINCIPLES REFERENCE FOR ENGINEERS

The Python Engineering Cookbook

From Interpreter Internals to Idiomatic, Production-Grade Python



FIRST EDITION

Jishnu Saha & **Mobasshir Bhuiya Shagor**

MBS PRESS

The Python Engineering Cookbook

First Edition · From Interpreter Internals to Idiomatic, Production-Grade Python

by Jishnu Saha and Mobasshir Bhuiya Shagor

Published by MBS Press.

Copyright © 2026 Jishnu Saha and Mobasshir Bhuiya Shagor. All rights reserved.

Printed for study and reference. No part of this book may be reproduced for commercial use without permission of the authors.

First edition, 2026 — including Testing, Data Structures & Algorithms, and Production Python.

Typeface & Production

Body set in Liberation Serif; display and labels in Liberation Sans; code in DejaVu Sans Mono. Source code is highlighted in a GitHub-light palette and runs on CPython 3.11+ unless noted.

Colophon

The animal on the cover is a python — a constrictor that, like the language, is admired for power expressed through economy of motion. The emblem is rendered as a coiled woodcut in keeping with the tradition of pairing technical books with engraved creatures.

Preface — How to read this cookbook

This book is built on a working engineer's study notes and organised into a progression: first *how Python runs* (the execution and object model), then *how it runs many things at once* (concurrency), then the *language mechanics* and *modern idioms* you reach for daily. Each topic follows the same rhythm: the mental model first, then the mechanism, then a code example, then — where it matters — a production note or a gotcha that bites people in real systems.

Code blocks are runnable on CPython 3.11+ unless stated. Three kinds of callouts recur: **GOTCHA** (a trap that causes real bugs), **IN PRODUCTION** (how the idea shows up in shipped systems), and **2026 UPDATE** (where the language has moved since older tutorials — most importantly, the optional removal of the GIL).

> ℹ️ **NOTE** Conventions: inline code looks like `this`; shaded blocks are runnable snippets. CPython is assumed throughout unless another implementation is named.

Contents

Part I — How Python Runs

1. Compiled or Interpreted? The Bytecode Model
2. CPython and the Implementation Zoo
3. Reference Counting and Garbage Collection

Part II — Concurrency and Parallelism

4. The Global Interpreter Lock (GIL)
5. Threading
6. Multiprocessing
7. AsyncIO
8. Choosing a Concurrency Model

Part III — The Object Model

9. Objects, Identity, Mutability and Copies

Part IV — Functions and Scope

10. Scope: LEGB, global and nonlocal
11. First-Class Functions: lambdas, closures, decorators
12. Iterators and Generators

Part V — Data Structures and Idioms

13. Comprehensions
14. Built-in Collections and the collections Module
15. Sorting and Timsort
16. type vs isinstance
17. Numbers, Division and Truthiness

Part VI — Resources and Errors

18. Context Managers
19. Exception Handling

Part VII — Modern Python Toolbox

20. Dataclasses and __slots__
21. Type Hints
22. Structural Pattern Matching
23. Modules and __name__
24. Environments and Packaging: pip, Poetry, uv

Part VIII — Testing

25. Testing with pytest
26. Fixtures, Mocking and Test Doubles

Part IX — Data Structures and Algorithms in Python

27. Key Data Structures
28. Key Algorithms

Part X — Python in Production

29. Debugging
30. Logging, Monitoring and OpenTelemetry
31. Caching
32. Queues and Task Processing
33. Security Essentials

34. Memory and the OS from Python

*Every chapter closes with an **Interview questions** box — likely prompts with crisp answers, to tie the material back to senior/staff interview prep.*

PART I

How Python Runs

1. Compiled or Interpreted? The Bytecode Model

Python is **both**. Source is first *compiled* to platform-independent **bytecode** (the `.pyc` files cached under `__pycache__/`), and that bytecode is then executed by the CPython *virtual machine*, which interprets it instruction by instruction. So the compile step is real, but it targets a virtual machine, not your CPU.

Why an interpreted (bytecode) model is convenient

- **Portability.** The bytecode targets the Python VM, not a specific CPU/OS, so the same program runs anywhere a compatible interpreter exists — no platform-specific binary to rebuild.
- **Fast edit–run loop.** No separate link/build step; you change a file and run it. Interactive shells (the Python REPL, Jupyter, a browser JS console) let you execute a line at a time, which is ideal for exploring an API or testing a snippet.
- **Dynamic typing.** Types are checked at runtime, so you don't annotate every declaration as in C/C++ (though optional type hints exist — see Chapter 21).
- **Managed memory.** The runtime allocates and frees objects for you (reference counting plus a cyclic collector — Chapter 3), removing whole classes of manual-memory bugs.

⚠️ **GOTCHA** A common misconception: "errors only surface at the failing line." Python compiles the *entire module* to bytecode before running it, so **syntax errors** are caught up front for the whole file. It is *runtime* errors (a bad key, a type mismatch) that halt execution at the offending line. Hot-reload tooling like a React dev server preserving app state on save is a *bundler/HMR* feature, not a property of "being interpreted."

ℹ️ **NOTE** A `.pyc` file is tied to the CPython *version* that produced it (the magic number in its header), so bytecode is portable across machines but not across interpreter versions.

INTERVIEW QUESTIONS

Q. Is Python interpreted or compiled?

A. Both: source is compiled to platform-independent bytecode (`.pyc`), which the CPython VM then interprets. The compile step is real, but it targets a virtual machine, not the CPU.

Q. When are syntax vs runtime errors detected?

A. Syntax errors are caught when the whole module compiles to bytecode, before any line runs; runtime errors (bad key, type mismatch) surface at the offending line during execution.

Q. Is a `.pyc` portable across machines and versions?

A. Across machines yes (bytecode is platform-independent), but not across interpreter versions — a magic number ties it to a specific CPython version.

2. CPython and the Implementation Zoo

"Python" is a language specification; **CPython** is its reference implementation, written in C, and by far the most widely deployed runtime. Because the VM and much of the standard library are C, CPython integrates tightly with C libraries — which is how Python reaches C-level speed for hot paths.

This is also why **built-ins beat hand-written Python**. A call like `sum(xs)` or `"".join(parts)` runs a tight C loop, while the equivalent `for` loop in Python pays per-iteration interpreter overhead. Reaching for the built-in is usually both clearer and faster.

Implementation	What it is	When you'd use it
CPython	Reference C implementation; the default `python`	Almost always; widest library support
PyPy	JIT-compiled Python	Long-running CPU-bound pure-Python code that needs speed
Jython / GraalPy	Runs on the JVM / GraalVM	Deep Java interop
MicroPython	Tiny build for microcontrollers	Embedded / IoT devices

ℹ️ **NOTE** Mental shortcut: when people say "the GIL," "refcounting," or "`.pyc`" they mean *CPython* specifics. PyPy and others make different choices.

INTERVIEW QUESTIONS

Q. What is CPython and why does it matter?

A. The reference C implementation and the default `python`. Most 'Python' specifics — the GIL, reference counting, `.pyc` — are CPython behaviours, not language guarantees.

Q. Why is `sum(xs)` faster than an equivalent Python loop?

A. Built-ins run as compiled C and avoid per-iteration interpreter overhead, so they beat hand-written Python doing the same work.

Q. When would you choose PyPy over CPython?

A. For long-running, CPU-bound, pure-Python code where the JIT pays off and you don't depend on C extensions PyPy handles poorly.

3. Reference Counting and Garbage Collection

Every CPython object carries a **reference count**: how many names/containers currently point at it. Binding or copying a reference increments it; rebinding or leaving scope decrements it. When the count hits zero, the object is freed immediately.

```
a = [1, 2, 3]      # object [1,2,3] -> refcount 1
b = [4, 5]         # object [4,5]   -> refcount 1

b = a              # [1,2,3] -> refcount 2 (b now points to it)
                   # [4,5]   -> refcount 0  -> freed at once
```

Reference counting is simple and frees memory promptly, but it cannot reclaim **reference cycles** (objects that point at each other). For those, CPython runs a separate **cyclic garbage collector** (the `gc` module) that periodically finds and collects unreachable cycles.

❖ **IN PRODUCTION** In latency-sensitive services you can tune or pause the cyclic collector (`gc.disable()`, `gc.freeze()`) around hot phases, then re-enable it — a known trick for cutting GC pauses in request hot paths and at process fork. Reach for it only after profiling.

ℹ️ **NOTE** Reference counting is exactly why the GIL exists. Read on.

INTERVIEW QUESTIONS

Q. How does CPython manage memory?

A. Primarily reference counting — objects are freed the moment their count hits zero — plus a cyclic garbage collector (`gc`) for reference cycles that refcounting can't reclaim.

Q. Why can't reference counting reclaim everything?

> **A.** It can't free reference cycles, where objects keep each other's counts above zero; the cyclic collector handles those.

Q. When would you touch the `gc` module in production?

> **A.** To cut GC pauses in latency-sensitive paths — e.g. `gc.disable()`/`gc.freeze()` around a request or at fork — but only after profiling shows GC is the cost.

PART II

Concurrency and Parallelism

4. The Global Interpreter Lock (GIL)

The **GIL** is a single mutex inside CPython that allows **only one thread to execute Python bytecode at a time**. It exists to protect the very reference counts from Chapter 3.

Why it exists

All threads in a process share memory, so they can read and write the same object's refcount concurrently. Consider two threads each running `b = a` on shared objects:

```
# Thread 1 and Thread 2 both adjust refcounts at the same time
# Expected: [1,2,3] refcount += 1   ;  [4,5] refcount -= 1
# Interleaved without protection:
#   [1,2,3] refcount becomes 3 instead of 2
#   [4,5]   refcount becomes -1  -> object freed while still in use
#   => memory corruption
```

Two ways to make refcount updates safe: put a fine-grained lock on *every* object (correct but slow, lots of lock traffic), or take *one* global lock around the interpreter. CPython historically chose the second — fast and simple for single-threaded code, which is the common case.

ℹ️ **NOTE** The GIL makes single-threaded code fast and C-extension integration simple. The cost: pure-Python threads cannot run *bytecode* in parallel across cores.

✦ **2026 UPDATE** Since Python 3.13 there is an experimental **free-threaded build** (PEP 703) that disables the GIL; with Python 3.14 (October 2025) that build became **officially supported but still opt-in** (PEP 779) — install it via `uv python install 3.14t`. CPU-bound Python threads can then scale near-linearly across cores. Caveats: single-threaded code runs ~5–10% slower (down from ~40% in 3.13), and if you import a C extension that hasn't declared itself thread-safe, CPython silently re-enables the GIL for the whole process — check `sys._is_gil_enabled()` after imports. Making the no-GIL build the default is a later phase (projected ~3.16+).

INTERVIEW QUESTIONS

Q. What is the GIL and why does it exist?

A. A single mutex that lets only one thread execute Python bytecode at a time; it makes reference-count updates safe without per-object locking.

Q. Does the GIL prevent all parallelism?

A. No — it's released during blocking I/O and inside many C extensions (e.g. NumPy), so I/O-bound threads overlap; it only blocks parallel execution of Python bytecode.

Q. What changed with the GIL in 2025–2026?

A. Python 3.14 makes the free-threaded (no-GIL) build officially supported but still opt-in; CPU-bound threads can scale across cores, at ~5–10% single-thread overhead and with C-extension caveats.

5. Threading

Under the GIL, Python threads run **concurrently, not in parallel**. They share one process and one memory space; the interpreter time-slices between them, switching roughly every 5 ms (`sys.getswitchinterval()` defaults to 0.005 s). Only one thread executes bytecode at any instant.

Crucially, the GIL is **released during blocking I/O** (network, disk, `time.sleep`). So while one thread waits on a socket, another runs — which is exactly why threading shines for **I/O-bound** work.

Use case: many I/O calls at once

Fetching ten URLs sequentially means waiting for each response before starting the next. A thread pool overlaps the waits:

```
import time, requests
from concurrent.futures import ThreadPoolExecutor

urls = ["https://httpbin.org", "https://restcountries.com",
        "https://catfact.ninja", "https://books.toscrape.com"]  # ... etc.

def fetch(url):
    try:
        r = requests.get(url, timeout=10)
        return f"[{r.status_code}] {url[:40]}"
    except Exception as e:
        return f"[ERROR] {url} -> {e}"

start = time.time()
with ThreadPoolExecutor(max_workers=10) as pool:   # all in flight together
    results = list(pool.map(fetch, urls))
print("\n".join(results))
print(f"Total: {time.time() - start:.2f}s")
```

Multiprocessing would also work here but is the wrong tool: it spins up extra cores that would only sit *idle waiting on the network* — wasteful when one core plus threads already overlaps the I/O.

Threading's limits

- **No CPU parallelism.** On CPU-bound work, GIL hand-offs add overhead, so threaded compute is often slower than plain sequential code.
- **Race conditions.** The GIL serialises bytecode but does not make multi-step operations atomic — see below.
- **Memory.** Each thread reserves a stack (often ~8 MB of address space). Tens of thousands of threads is not viable; that scale calls for asyncio (Chapter 7).

The classic race condition

Even with the GIL, `counter += 1` is **not atomic**. It compiles to several bytecodes — load, add, store — and a thread switch can land in the middle:

```
# counter starts at 5
# counter += 1   compiles to roughly:
#   LOAD counter   -> 5
#   LOAD 1
#   ADD            -> 6   (thread may be paused right here)
#   STORE counter

# T1 loads 5, is paused before STORE
# T2 loads 5, adds, stores 6
```

```
# T1 resumes, stores 6
# Two increments happened; counter is 6, not 7   -> lost update
```

The fix is to make the update mutually exclusive with a lock (a `with lock:` block):

```
from threading import Lock
lock = Lock()

with lock:          # only one thread in this block at a time
    counter += 1
```

❖ **IN PRODUCTION** This is the bug behind "the numbers don't add up under load" — double-spend, miscounted inventory, lost metrics. Guard shared mutable state with a `Lock`, or avoid sharing it (give each worker its own accumulator and merge at the end).

✦ **2026 UPDATE** On the free-threaded build, this matters *more*: code that was "accidentally safe" because the GIL serialised it can now truly race. The advice for 3.14 is explicit — protect shared mutable state with `threading.Lock` rather than relying on built-in types' internal locks.

INTERVIEW QUESTIONS

Q. When should you use threads in Python?

A. For I/O-bound work with modest fan-out, especially with blocking libraries; the GIL releases on I/O so the waits overlap.

Q. Is `counter += 1` thread-safe under the GIL?

A. No — it is several bytecodes (load/add/store) and a switch can interleave them, losing updates. Guard it with a `Lock`.

Q. Why avoid threads for CPU-bound work?

A. The GIL serialises bytecode, so threads can't use multiple cores and hand-offs add overhead — often slower than sequential.

6. Multiprocessing

To use many CPU cores for Python computation, give each worker its own interpreter and its own GIL. `ProcessPoolExecutor(max_workers=N)` asks the OS to create `N` independent processes; each is a fresh Python runtime with its **own interpreter, own GIL, own memory, and (as the OS schedules) its own core**. That is true parallelism.

How children start: spawn vs fork

Start method	Platform default*	What the child does
spawn	Windows, macOS	Boots a blank Python, re-imports your script top-to-bottom, then takes work
fork	Linux (historically)	Child is a memory copy of the parent: functions already loaded, starts immediately

*See the 2026 note below — the Linux default is moving.

Why the __main__ guard is mandatory

On **spawn**, the child re-imports your module, so any *top-level* code runs again in every child. Put process-creation inside the guard or you get an infinite spawn loop that crashes:

```
# script.py
def run():
    ...                       # the parallel work
```

```
print("hello")                # runs in EVERY child on spawn -- be careful

if __name__ == "__main__":    # True only in the process you launched directly
    run()                     # child imports set __name__ = "script", so this is
skipped
```

⚠️ **GOTCHA** Without the guard, `run()` at top level re-executes on every re-import, each child spawns more children, and the program melts down. This is the single most common multiprocessing mistake.

The bottleneck: inter-process communication

Processes don't share memory, so every argument and return value crosses a process boundary: it is **pickled** (serialised), piped, and unpickled. That has real cost.

- If the data is large or the task is tiny, IPC overhead can erase the parallelism gains.
- Rule of thumb: each task should do at least ~100 ms of CPU work to be worth a separate process.

```
import math, os, time
from concurrent.futures import ProcessPoolExecutor

def heavy(n):
    return sum(math.sqrt(i) * math.log(i + 1) for i in range(1, n + 1))

numbers = [100_000_000, 200_000_000, 30_000_000]

if __name__ == "__main__":
    start = time.time()
    with ProcessPoolExecutor(max_workers=os.cpu_count() - 1) as pool:
        list(pool.map(heavy, numbers))      # spread across cores
    print(f"Multiprocessing: {time.time() - start:.2f}s")
```

✦ **2026 UPDATE** fork is unsafe in a process that already has threads (it can deadlock). CPython 3.12+ warns about this, and recent versions are moving the Linux default *away from fork* (toward forkserver). For portable, predictable behaviour, set it explicitly: `multiprocessing.set_start_method("spawn")`. Python 3.14 also adds first-class **subinterpreters** (`concurrent.interpreters`, PEP 734): parallel isolated interpreters within one process — lighter than processes, no pickling for the interpreter itself.

INTERVIEW QUESTIONS

Q. How does multiprocessing achieve true parallelism?

A. Each worker is a separate process with its own interpreter and GIL, so they run Python bytecode in parallel across cores.

Q. Why is the `if __name__ == '__main__'` guard required?

A. On spawn platforms each child re-imports the module; without the guard, top-level process-creation re-runs in every child and spawns infinitely.

Q. What is multiprocessing's main cost?

A. IPC: arguments and results are pickled, piped, and unpickled. For tiny tasks or large payloads it can erase the gains — aim for at least ~100 ms of CPU work per task.

7. AsyncIO

Threading uses **preemptive** switching — the runtime can pause a thread at any time. **AsyncIO** uses **cooperative** switching — a coroutine voluntarily yields control at an `await`. One thread, one event loop, but thousands of overlapping I/O operations.

What await actually does

- When execution hits `await` on something that must wait (a sleep, a socket read), the coroutine *pauses* and hands that wait to the OS.
- Control returns to the **event loop**, which runs the next ready task while the OS watches the pending I/O.
- When the I/O completes, the OS marks it done; the task moves from the *waiting pool* back to the *ready queue*, and the loop resumes it from just after its `await`.

The event loop

The loop is just a Python object created by `asyncio.run()` and destroyed when it returns. Conceptually it is a `while` loop that keeps asking the OS "anything ready?"

```
# Simplified mental model of asyncio.run(main()):
loop = asyncio.new_event_loop()
loop.run_until_complete(main())
loop.close()

# The loop itself:
while tasks_exist:
    for task in os.poll_for_completed_io():   # ask the OS what's ready
        task.resume()
```

Building blocks

- **async def** — calling it runs no code; it returns a coroutine object (a paused function).
- **asyncio.run(main())** — the one entry point from sync code: creates the loop, runs the coroutine, tears the loop down.
- **asyncio.create_task(coro)** — schedules the coroutine on the ready queue and returns a Task immediately; it does not run it yet.
- **await x** — if x is a coroutine, it starts running inline; if x is a Task, control goes to the event loop and the current coroutine resumes when x finishes. await also yields the awaited result.

Concurrency comes from create_task, not from await alone

Awaiting sleeps one after another is **sequential**:

```
# SEQUENTIAL -- about 3 seconds total
async def main():
    await asyncio.sleep(1)
    await asyncio.sleep(1)
    await asyncio.sleep(1)
```

Scheduling them as tasks first lets them overlap. While the first awaited task sleeps, the loop starts the others, so the whole thing finishes in about one second:

```
# CONCURRENT -- about 1 second total
async def main():
    t1 = asyncio.create_task(asyncio.sleep(1))
    t2 = asyncio.create_task(asyncio.sleep(1))
    t3 = asyncio.create_task(asyncio.sleep(1))
    await t1    # yields to the loop; t2 and t3 are already running
    await t2
    await t3

asyncio.run(main())
```

⚠️ **GOTCHA** If `main()` returns while tasks are still running, the loop is torn down and those tasks may be *cancelled*. Awaiting only `t1` below lets t2/t3 start but not necessarily finish:

```
async def fetch(n):
    print("start", n); await asyncio.sleep(n); print("done", n)

async def main():
    t1 = asyncio.create_task(fetch(1))
    t2 = asyncio.create_task(fetch(2))
    t3 = asyncio.create_task(fetch(3))
    await t1        # prints: start 1 / start 2 / start 3 / done 1   (then loop ends)

# Correct: wait for all of them together
async def main():
    await asyncio.gather(fetch(1), fetch(2), fetch(3))   # all complete
```

Prefer `asyncio.gather(...)` (or, in 3.11+, `asyncio.TaskGroup`) to run a batch and guarantee every member completes.

ℹ️ **NOTE** Two terms worth keeping straight. **Ready queue**: tasks ready to run, waiting for the loop to pick them. **Waiting pool**: tasks parked on I/O or a timer. **Event-loop latency** is the gap between a task becoming ready and the loop getting around to it — which is why one CPU-heavy coroutine (no `await`) stalls the entire loop.

❖ **IN PRODUCTION** This is the engine under modern async web stacks (FastAPI, ASGI servers, aiohttp). One process can hold tens of thousands of open connections cheaply — but never call a *blocking* function inside a coroutine; offload it with `loop.run_in_executor(...)` or `asyncio.to_thread(...)` so the loop keeps turning.

INTERVIEW QUESTIONS

Q. How does asyncio differ from threading?

A. Threading is preemptive (switch anytime); asyncio is cooperative — a coroutine yields only at `await` — running thousands of overlapping I/O ops in one thread.

Q. Does `create_task` run the coroutine immediately?

A. No — it schedules it on the event loop's ready queue and returns a Task; it runs when the loop next gets control, e.g. at an `await`.

Q. What if you call a blocking/CPU-heavy function in a coroutine?

A. It stalls the whole event loop, since nothing yields; offload it with `asyncio.to_thread()` or `loop.run_in_executor()`.

8. Choosing a Concurrency Model

Workload	Best tool	Why
I/O-bound, modest fan-out (tens)	Threading	Simple; works with blocking libraries (`requests`, many DB drivers); GIL releases on I/O
I/O-bound, massive fan-out (thousands)	AsyncIO	One thread holds thousands of connections; far lower memory than threads
CPU-bound	Multiprocessing	True parallelism across cores, each with its own GIL
CPU-bound, no/compatible C-ext	Free-threaded build	3.14+ opt-in: real thread parallelism without process/IPC overhead

Decision rules of thumb

- **Waiting on the network/disk?** You are I/O-bound → threading (simple) or asyncio (at scale).
- **Burning CPU (math, parsing, image work)?** You are CPU-bound → multiprocessing (or the free-threaded build, if your dependencies support it).
- **Stuck with blocking libraries you can't rewrite?** Threading, because asyncio needs async-native libraries to deliver its benefit.
- **Memory efficiency at very high concurrency matters?** AsyncIO.

❖ **IN PRODUCTION** Real systems mix these: an async web server (asyncio) that offloads a CPU-heavy step to a `ProcessPoolExecutor`, while a background thread tails a log. Match the tool to each task, not to the whole app.

INTERVIEW QUESTIONS

Q. I/O-bound vs CPU-bound — how do you choose?

A. I/O-bound: threading (modest scale) or asyncio (thousands of connections). CPU-bound: multiprocessing, or the 3.14 free-threaded build if dependencies allow.

Q. Why asyncio over threads at very high concurrency?

A. Each thread reserves megabytes of stack, so tens of thousands of threads is infeasible; asyncio holds the same connections in one thread with far less memory.

Q. Can you mix models?

A. Yes — commonly an async server that offloads CPU-heavy steps to a process pool. Match the tool to each task, not the whole app.

PART III

The Object Model

9. Objects, Identity, Mutability and Copies

In Python every value is an object with an identity (its address), a type, and a value. Variables are **names bound to objects**, not boxes holding bytes — a distinction that explains mutability, `is` vs `==`, and copy semantics.

Mutable vs immutable

- **Mutable** — value can change in place; the identity stays the same. Lists, dicts, sets.

```
a = []              # id(a) = 1293...
a.append(4)         # id(a) = 1293...  (same object, mutated)
```

- **Immutable** — value cannot change; "modifying" rebinds the name to a new object (or errors). int, float, str, tuple, frozenset.

```
a = 4               # id 3232...
a += 8              # id 9389...  (new int object)
a = (2, 4, 5)
a[0] = 9            # TypeError: tuples are immutable
```

Interning: when equal values share identity

CPython **reuses** some small immutable objects to save memory, so two equal values may share one identity — but this is an **optimisation, not a guarantee**.

- Small integers in the range `-5` to `256` are cached, so `c = 4; d = 4` gives `c is d == True`.
- Short string literals are often *interned*, so `"hello" is "hello"` is usually `True`.
- Equal tuples may share identity only through compile-time *constant folding* within one code block; created separately at runtime they are usually distinct objects. Never rely on it.

```
a, b = "hello", "hello"
a == b              # True  (equal value)
a is b              # True  (cached) -- but do not depend on this

x = (2,3,4,5,6,7,8,9,0,3,4,5)
y = (2,3,4,5,6,7,8,9,0,3,4,5)
x is y              # typically False for larger tuples
```

⚠️ **GOTCHA** Always compare values with `==` and reserve `is` for identity checks against singletons like `None` (`x is None`). Using `is` for value comparison "works" on cached small ints and breaks on everything else — a textbook heisenbug.

is vs ==

- `==` — equality; compares value via the object's __eq__.
- **is** — identity; do the two names point to the same object in memory?

Shallow vs deep copy

- **Shallow copy** (copy.copy) — new outer object, but nested objects are still shared by reference.
- **Deep copy** (copy.deepcopy) — new outer object and recursively new copies of everything nested.

```python
import copy
original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
shallow[0].append(99)         # also changes original[0] -> nested list is shared

deep = copy.deepcopy(original)
deep[0].append(99)            # original untouched
```

❖ **IN PRODUCTION** Shallow-copy bugs are common when caching or returning "snapshots" of config/state: callers mutate a nested list and silently corrupt the source. If a snapshot must be independent, `deepcopy` it (or rebuild it from immutable parts).

INTERVIEW QUESTIONS

Q. `is` vs `==`?

A. `==` compares value (via `__eq__`); `is` compares identity (same object). Use `is` only for singletons like `None`.

Q. Why does `a is b` hold for some equal ints but not others?

A. CPython caches small ints (−5..256) and interns short strings as an optimisation; it's not guaranteed, so never use `is` for value comparison.

Q. Shallow vs deep copy?

A. Shallow (`copy.copy`) makes a new outer object but shares nested ones; deep (`copy.deepcopy`) copies recursively. Use deepcopy when a snapshot must be independent.

PART IV

Functions and Scope

10. Scope: LEGB, global and nonlocal

Name lookup follows the **LEGB** rule, innermost first: **Local** → **Enclosing** function → **Global** (module) → **Built-in**. Reading an outer name is always allowed; *rebinding* one is not, unless you say so.

```
x = 10                  # global
def f():
    print(x)            # reads global x -> 10
f()

count = 5
def bad():
    count = count + 1   # UnboundLocalError: assignment makes 'count' local,
                         # but it's read before assignment
bad()
```

Assigning to a name anywhere in a function makes it **local for the whole function**; reading it before assignment then fails. Declare intent explicitly:

- **global x** — bind the module-level variable.
- **nonlocal x** — bind the nearest enclosing function's variable.

```
count = 5
def inc():
    global count
    count += 1          # now updates the module-level count
inc(); print(count)     # 6

def outer():
    n = 0
    def inner():
        nonlocal n      # updates outer()'s n
        n += 1
    inner(); print(n)   # 1
outer()
```

ℹ️ **NOTE** Two edge cases worth memorising. `global x` with `x = 1` will *create* the global if absent; but `x += 1` on an absent global raises `NameError`. And `nonlocal` with no matching enclosing binding is a *SyntaxError* at compile time.

INTERVIEW QUESTIONS

Q. What is the LEGB rule?

A. Name lookup order, innermost first: Local → Enclosing function → Global (module) → Built-in.

Q. Why does `count = count + 1` raise UnboundLocalError for a global `count`?

A. Assigning anywhere in a function makes the name local for the whole function, so reading it first fails; use `global` (or `nonlocal`) to rebind the outer one.

Q. `global` vs `nonlocal`?

A. `global` rebinds a module-level name; `nonlocal` rebinds the nearest enclosing function's name.

11. First-Class Functions: lambdas, closures, decorators

Functions are objects: you can pass them, return them, and store them. Three idioms build on this.

Lambdas

A **lambda** is a small anonymous, single-expression function whose value is returned automatically. Use it for short, throwaway callables (a sort key, a `map` function); name a `def` for anything non-trivial.

```
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, nums))     # [2, 4, 6, 8]
```

Closures

A **closure** is a function that remembers variables from its enclosing scope even after that scope has returned. It is what makes decorators (and many factory functions) work.

```
def adder(x):
    def add(y):
        return x + y          # remembers x
    return add

add5 = adder(5)
add5(3)                       # 8
```

Decorators

A **decorator** wraps a function to add behaviour without editing it. You receive a function, define a wrapper that calls it, and return the wrapper.

```
def add_logging(func):
    def wrapper(*args, **kwargs):       # accept any signature
        print("before")
        result = func(*args, **kwargs)
        print("after")
        return result
    return wrapper

@add_logging                            # sugar for: greet = add_logging(greet)
def greet():
    print("hello")

greet()    # before / hello / after
```

The `@add_logging` line is exactly `greet = add_logging(greet)` — cleaner syntax for the same wrapping.

> ⚠ **GOTCHA** A bare wrapper **hides the original function's identity** (`greet.__name__` becomes `"wrapper"`), which breaks introspection, docs, and some frameworks. Always preserve metadata with `functools.wraps`:

```
from functools import wraps

def add_logging(func):
    @wraps(func)                        # copies __name__, __doc__, signature
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

*args and **kwargs

These let a function (or wrapper) accept any arguments. `*args` collects positional arguments into a tuple; `**kwargs` collects keyword arguments into a dict. Essential for writing general-purpose decorators.

❖ **IN PRODUCTION** A `functools.lru_cache` decorator gives you a memoisation cache for free — a one-line speedup for pure functions with repeated inputs (recursive computations, expensive lookups):

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)
```

The mutable-default-argument trap

A default argument is evaluated **once**, when the function is defined — not on each call. A mutable default is therefore shared across calls:

```
def append_bad(item, bucket=[]):     # WRONG: one list, reused forever
    bucket.append(item)
    return bucket

append_bad(1)    # [1]
append_bad(2)    # [1, 2]  <-- surprise: same list

def append_ok(item, bucket=None):    # RIGHT
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

⚠ **GOTCHA** This is one of Python's most famous bugs. Default to `None` and build the mutable object inside the function.

INTERVIEW QUESTIONS

Q. What is a closure and where is it used?

A. A function that remembers variables from its enclosing scope after that scope returns; it underlies decorators and factory functions.

Q. What does a decorator do, and why `functools.wraps`?

A. It wraps a function to add behaviour without editing it (`@d` is `f = d(f)`); `wraps` copies the original's name/doc/signature so introspection and frameworks still work.

Q. Why is a mutable default argument dangerous?

A. Defaults evaluate once at definition, so `def f(x, acc=[])` shares one list across calls; default to `None` and build the object inside.

12. Iterators and Generators

Iterables vs iterators

- **Iterable** — an object with __iter__() that returns an iterator (lists, dicts, strings).
- **Iterator** — implements __iter__() (returns itself) and __next__() (returns the next value, raising StopIteration when exhausted). It remembers its position.

So a list is *iterable* but not itself an *iterator*; `iter(my_list)` produces the iterator that a `for` loop drives.

Generators

A **generator** is the easy way to build an iterator: a function that `yield`s values one at a time, pausing after each `yield` and resuming on the next request. It produces values lazily instead of building a whole list.

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i           # pause here, return i; resume on next()
        i += 1

g = count_up_to(3)        # nothing runs yet
next(g)                   # 1
next(g)                   # 2

squares = (x*x for x in range(5))   # generator expression
```

- **Lazy and memory-cheap.** It holds one value at a time, not the whole sequence — the difference between returning a million-element list and streaming a million values.

```
def all_numbers():            # builds a 1,000,000-element list in memory
    return [i for i in range(1_000_000)]

def stream_numbers():        # yields one at a time; constant memory
    for i in range(1_000_000):
        yield i
```

- **Single-use.** A generator is exhausted after one pass; iterate again and you get nothing. Rebuild it if you need a second pass.

❖ **IN PRODUCTION** Generators are the standard pattern for streaming large files and data pipelines: `for line in file:` reads one line at a time rather than `file.readlines()` loading the whole file. Chain generators to build memory-flat ETL stages. `yield from sub_gen` delegates to another generator, and `gen.send(value)` can feed values back in.

INTERVIEW QUESTIONS

Q. Iterable vs iterator?

A. An iterable has `__iter__` returning an iterator; an iterator implements `__iter__` and `__next__` and tracks position. A list is iterable, not an iterator.

Q. Why use a generator?

A. Lazy, constant-memory production of values — stream a million items instead of building a million-element list; ideal for large files and pipelines.

Q. A common generator gotcha?

A. They're single-use: once exhausted, iterating again yields nothing. Rebuild it for a second pass.

PART V

Data Structures and Idioms

13. Comprehensions

A comprehension builds one iterable from another in a single, readable expression. Four forms exist:

```
# list
squares = [x*x for x in range(5)]
evens   = [i for i in range(10) if i % 2 == 0]          # with filter
labels  = ["even" if i % 2 == 0 else "odd" for i in range(5)]   # with if/else
pairs   = [(x, y) for x in [1,2] for y in [3,4]]        # nested loops

# dict
even_sq = {x: x*x for x in range(6) if x % 2 == 0}      # {0:0, 2:4, 4:16}

# set
uniq    = {c.lower() for c in "Hello"}

# generator (lazy -- parentheses, not brackets)
gen     = (i*i for i in range(5))
```

ℹ️ **NOTE** Note the placement: a *filter* ("include only if") goes *after* the loop (`... for x in xs if cond`); an *if/else expression* ("choose value") goes *before* the loop. Keep comprehensions one level deep; beyond that a plain loop reads better.

INTERVIEW QUESTIONS

Q. Where do a filter and an if/else go in a comprehension?

A. A filter goes after the loop (`x for x in xs if cond`); an if/else value-choice goes before it (`a if cond else b for x in xs`).

Q. List comprehension vs generator expression?

A. Brackets build the whole list in memory; parentheses make a lazy generator that yields on demand — prefer the generator for large or streamed data.

14. Built-in Collections and the collections Module

Sets

- **Hash-based** — membership tests and insertions are O(1) on average.
- **Unordered, de-duplicated** — duplicates are dropped; iteration order is not guaranteed (don't rely on it).

Dictionary keys must be hashable

A dict key must be hashable: an *immutable* object with a stable `__hash__` and a consistent `__eq__`. So `int`, `str`, `frozenset`, and tuples-of-hashables work as keys; `list`, `dict`, and `set` do not.

```
ok = {
    1: "int",
    "name": "str",
```

```
    (2, 3): "tuple of hashables",
    frozenset({1, 2}): "frozenset",
}

bad = { [1, 2]: "x" }        # TypeError: unhashable type: 'list'
bad = { (1, [2]): "x" }      # TypeError: tuple holds a list -> unhashable
```

- Keys are unique: assigning an existing key overwrites its value.
- The hash must stay constant for the object's lifetime, or you can never find the entry again — which is exactly why mutable objects are banned as keys.

range is lazy, reusable, and indexable

A `range` is *not* a generator. It is a lazy arithmetic sequence object: it stores only start/stop/step, yet supports indexing, `len()`, membership, and — unlike a generator — **repeated** iteration.

```
r = range(5)
r[2]          # 2     (indexable)
len(r)        # 5
list(r); list(r)   # reusable -- a generator would be empty the second time
```

The collections module

Type	What it gives you	Typical use
`Counter`	Counts hashable items; `.most_common(k)`	Word/event frequencies, top-k
`defaultdict`	Auto-creates a default value for missing keys	Grouping; adjacency lists; tallies
`deque`	O(1) appends/pops at both ends	Queues, sliding windows, BFS frontiers
`namedtuple`	Tuple with named fields	Lightweight immutable records

```
from collections import Counter, defaultdict, deque

Counter("mississippi").most_common(2)     # [('i', 4), ('s', 4)]

groups = defaultdict(list)
for word in ["ant", "art", "bee"]:
    groups[word[0]].append(word)          # no key-exists check needed

q = deque([1, 2, 3])
q.appendleft(0); q.pop()                  # O(1) at both ends
```

❖ **IN PRODUCTION** Reach for `defaultdict(list)` / `Counter` instead of hand-rolling "if key not in d" logic — fewer lines, fewer bugs, and they signal intent. Use `deque` (not a list) whenever you pop from the front: `list.pop(0)` is O(n).

INTERVIEW QUESTIONS

Q. Why must dict keys be hashable, and what qualifies?

A. The dict locates entries by a stable hash, so keys must be immutable with consistent `__hash__`/`__eq__`: int, str, frozenset, tuples of hashables — not list/dict/set.

Q. Membership cost: set/dict vs list?

A. O(1) average for set/dict (hash lookup) vs O(n) for a list (linear scan).

Q. When use `deque` over a list?

A. When popping/appending at the front: `deque` is O(1) at both ends, while `list.pop(0)` is O(n).

15. Sorting and Timsort

Python's `sorted()` and `list.sort()` use **Timsort** — a hybrid of merge sort and insertion sort that exploits already-ordered runs. It is **stable** (equal elements keep their input order) and runs in O(n log n) worst case, O(n) on nearly-sorted data.

Sorting by a key

Pass `key=` a function that maps each element to the value you want to sort on. Return a *tuple* to sort by several criteria at once; tuples compare element by element, left to right.

```
nums = [1, 1, 2, 2, 2, 3]
freq = {1: 2, 2: 3, 3: 1}

# Sort by frequency ascending, then by value ascending:
nums.sort(key=lambda x: (freq[x], x))
# keys: 1->(2,1) 2->(3,2) 3->(1,3)
# ordered by tuple: (1,3)=3, (2,1)=1, (3,2)=2  -> [3, 1, 1, 2, 2, 2]
```

To reverse one field while keeping another ascending, negate a numeric key: `key=lambda x: (-freq[x], x)`. Stability means you can also sort in passes (least-significant key first).

❖ **IN PRODUCTION** Because Timsort is stable, multi-level sorting "just works": sort by secondary key, then by primary key, and ties preserve the secondary order. This is the backbone of leaderboard, ranking, and report ordering logic.

INTERVIEW QUESTIONS

Q. What algorithm does Python's sort use?

A. Timsort — a stable, adaptive merge/insertion hybrid; O(n log n) worst case, O(n) on nearly-sorted data.

Q. How do you sort by several criteria?

A. Return a tuple from `key`; tuples compare left to right. Negate a numeric field to reverse just that key.

Q. Why does stability matter?

A. It preserves the order of equal elements and lets you sort in passes — the basis of multi-level ranking and report ordering.

16. type vs isinstance

	`type(obj)`	`isinstance(obj, X)`
Returns	the exact class object	a boolean
Inheritance	ignored (exact class only)	respected (subclasses count)
Args	just the object	object plus a class or tuple of classes

```
class A: pass
class B(A): pass
b = B()

type(b) is B            # True
type(b) is A            # False  -- exact class only

isinstance(b, B)        # True
```

```
isinstance(b, A)        # True   -- subclass counts
isinstance(b, (A, int)) # True   -- tuple = "any of these"
```

❖ **IN PRODUCTION** Prefer `isinstance` for type checks — it honours inheritance, so code keeps working when someone subclasses your type. Reserve exact `type(x) is T` for the rare case where a subclass genuinely must be rejected. (Better still, lean on duck typing or `typing.Protocol` — Chapter 21.)

INTERVIEW QUESTIONS

Q. `type()` **vs** `isinstance()`**?**

A. `type(x) is T` checks the exact class and ignores inheritance; `isinstance(x, T)` is True for subclasses too and accepts a tuple of types.

Q. Which to prefer, and why?

A. `isinstance`, because it honours inheritance so code survives subclassing; reserve exact `type` checks for when a subclass must be rejected.

17. Numbers, Division and Truthiness

Floor division //

The `//` operator is **floor** division — the largest integer *less than or equal to* the true quotient. It does *not* simply chop off the decimal, which matters for negatives:

```
6 // 132     # 0     (6/132 = 0.045  -> floor = 0)
-6 // 132    # -1    (-0.045 -> floor = -1, NOT 0)

import math
math.floor(-3.5)   # -4   (floor: toward negative infinity)
int(-3.5)          # -3   (int: truncates toward zero)
```

⚠ **GOTCHA** Floor division and `int()` disagree on negatives. `//` and `math.floor` go toward negative infinity; `int()` truncates toward zero. Off-by-one bugs in pagination, bucketing, and time math usually trace back to this.

Falsy values

These objects are **falsy** (treated as `False` in a boolean context); everything else is truthy:

Falsy value	Type / note
`False`	bool
`None`	NoneType
`0` `0.0`	numeric zero
`""`	empty string
`[]` `()` `{}` `set()`	empty containers (note: `[]` is truthy in JavaScript)

⚠ **GOTCHA** "Empty is falsy" enables clean checks like `if items:`, but it bites when `0` or `""` is a *valid* value. To distinguish "absent" from "zero/empty," test `if x is None:` explicitly rather than `if not x:`.

INTERVIEW QUESTIONS

Q. Why is `-6 // 132 == -1`**, not 0?**

A. `//` is floor division (toward negative infinity), not truncation; `int(-0.045)` truncates to 0 but `math.floor(-0.045)` is −1.

Q. Which values are falsy?

A. `False`, `None`, numeric zero, empty string, and empty containers (`[]`, `()`, `{}`, `set()`); everything else is truthy.

Q. Why prefer `if x is None` over `if not x`?

A. `not x` is also true for 0/''/empty, so it can't tell 'absent' from a valid zero or empty value.

PART VI

Resources and Errors

18. Context Managers

A **context manager** guarantees setup and cleanup around a block — opening then closing files, acquiring then releasing locks, beginning then committing/rolling-back transactions — even if the block raises. You use it with `with`.

```
with open("data.txt") as f:
    process(f)
# f is closed here automatically -- even if process() raised

# Roughly what 'with' does:
mgr = open("data.txt")
f = mgr.__enter__()
try:
    process(f)
finally:
    mgr.__exit__(exc_type, exc_value, traceback)
```

Any object implementing `__enter__()` and `__exit__()` is a context manager. The cleanest way to write one is the `@contextmanager` decorator: code before `yield` is the setup (`__enter__`), code after it is the teardown (`__exit__`), and the yielded value is bound after `as`.

```
from contextlib import contextmanager

@contextmanager
def open_file(path):
    f = open(path, "w")
    try:
        yield f                # value bound to 'as f'
    finally:
        f.close()              # runs even if the body raises

with open_file("test.txt") as f:
    f.write("hello")
```

⚠ **GOTCHA** Wrap the `yield` in `try/finally`. Without it, an exception in the `with` body skips your cleanup and you leak the resource (an unclosed file, a held lock, an open transaction).

❖ **IN PRODUCTION** The same object can serve double duty: a DB connection works standalone *and* as a context manager, because it also implements `__enter__/__exit__`. `with conn:` then scopes a transaction — commit on success, roll back on exception.

INTERVIEW QUESTIONS

Q. What problem do context managers solve?

A. Guaranteed setup/teardown around a block — closing files, releasing locks, committing/rolling back — even if the block raises.

Q. How do you write one with `@contextmanager`?

A. Code before `yield` is setup, the yielded value binds after `as`, code after `yield` is teardown; wrap the `yield` in try/finally so cleanup runs on exceptions.

19. Exception Handling

The full shape is `try / except / else / finally`:

```
try:
    risky()
except ValueError as e:
    handle(e)          # runs only on ValueError
except (KeyError, IndexError):
    handle_lookup()    # one handler for several types
else:
    on_success()       # runs only if try finished with NO exception
                       # (and no return/break/continue in try)
finally:
    cleanup()          # ALWAYS runs
```

- **except** handles specific exception types — catch the narrowest type you can, not a bare except.
- **else** holds the success-only logic, keeping it out of the try so you don't accidentally catch its errors.
- **finally** always runs — cleanup that must happen regardless of success, exception, or early return.

⚠ **GOTCHA** Never `return` from `finally`: it silently overrides a `return` (or even a propagating exception) from the `try` block.

```
def test():
    try:
        return 1
    finally:
        return 2       # swallows the '1' -- test() returns 2.  Don't do this.
```

Custom exceptions

Define a domain exception hierarchy by subclassing `Exception`. Callers can then catch your base type and let specific subtypes carry detail — far better than raising bare `Exception` or stuffing errors into return values.

```
class PaymentError(Exception): pass
class CardDeclined(PaymentError): pass
class InsufficientFunds(PaymentError): pass

try:
    charge(card, amount)
except PaymentError as e:        # catches any payment failure
    log.warning("payment failed: %s", e)
```

❖ **IN PRODUCTION** Catch narrowly, raise meaningfully, and preserve the chain. When re-raising, use `raise NewError(...) from e` so the original cause stays in the traceback — invaluable when debugging production incidents.

INTERVIEW QUESTIONS

Q. What do `else` and `finally` add to try/except?

A. `else` runs only when no exception occurred (keeps success logic out of the try); `finally` always runs, for cleanup regardless of outcome.

Q. Why never `return` from `finally`?

A. It silently overrides a return or a propagating exception from the try block, hiding bugs.

Q. How do you keep the original error when re-raising?

A. `raise NewError(...) from e`, preserving the cause chain in the traceback.

PART VII

Modern Python Toolbox

20. Dataclasses and __slots__

A **dataclass** auto-generates the boilerplate for classes that mainly hold data: `__init__`, `__repr__`, and `__eq__` are written for you from the field declarations.

```
from dataclasses import dataclass, field

@dataclass
class Point:
    x: int
    y: int = 0                      # default value

p = Point(3, 4)
p                                   # Point(x=3, y=4)   (free __repr__)
Point(3, 4) == Point(3, 4)          # True              (free __eq__)
```

Defaults, mutable fields, and frozen instances

- **default_factory** — for a mutable default (list/dict/set), use field(default_factory=list); a plain mutable default is a TypeError here, by design (Chapter 11's trap).
- **frozen=True** — makes instances immutable and hashable, so they can be dict keys or set members.
- **order=True** — generates comparison methods, so instances sort by their fields as a tuple.

```
@dataclass(frozen=True, order=True)
class Money:
    amount: int
    currency: str = "USD"
    tags: list = field(default_factory=list)   # safe mutable default

sorted([Money(5), Money(2)])        # order=True -> sorts by (amount, currency)
{Money(5), Money(2)}                # frozen -> hashable -> usable in a set
```

__slots__ for memory

By default each instance stores its attributes in a per-instance `__dict__`. Declaring `__slots__` replaces that dict with a fixed array of fields — less memory per object and faster attribute access, at the cost of disallowing new attributes. It pays off when you create millions of small objects.

```
class PointSlots:
    __slots__ = ("x", "y")          # no per-instance __dict__
    def __init__(self, x, y):
        self.x, self.y = x, y

# Python 3.10+: dataclasses can do this directly
@dataclass(slots=True)
class FastPoint:
    x: int
    y: int
```

❖ **IN PRODUCTION** Dataclasses are the default choice for config objects, API request/response models, and value objects. For validation and serialisation on top (web APIs, settings), teams reach for `pydantic`, whose models follow the same field-declaration style.

INTERVIEW QUESTIONS

Q. What does `@dataclass` generate?
A. `__init__`, `__repr__`, and `__eq__` from the fields; options add ordering (`order=True`), immutability (`frozen=True`), and slots.

Q. How do you give a dataclass a mutable default safely?
A. `field(default_factory=list)` — a bare mutable default is a TypeError by design, dodging the shared-default trap.

Q. What does `__slots__` buy you?
A. It replaces the per-instance `__dict__` with a fixed field array — less memory, faster attribute access — but disallows new attributes.

21. Type Hints

Type hints annotate the *intended* types of variables, parameters, and returns. Python does **not** enforce them at runtime — they are for humans and for static checkers like `mypy` / `ty` — but they catch a large class of bugs before the code ever runs and make large codebases navigable.

```
def greet(name: str, times: int = 1) -> str:
    return (f"hi {name} " * times).strip()

from typing import Optional
def find_user(uid: int) -> Optional[dict]:   # may return a dict or None
    ...

# Modern (3.10+) syntax for unions and built-in generics:
def find_user(uid: int) -> dict | None: ...
scores: list[int] = []
index: dict[str, int] = {}
```

- **Optional[T] / T | None** — the value may be that type or None; checkers force you to handle the None case.
- **Protocol** — structural ("duck") typing: anything with the right methods qualifies, no inheritance required.
- **TypedDict / dataclasses** — give shape to dict-like records.

❖ **IN PRODUCTION** On a team or a long-lived service, hints plus a checker in CI are a force multiplier: refactors become safe, IDE autocomplete gets sharp, and "`None` where a dict was expected" is caught at review time, not at 3 a.m. Annotate public function signatures first; let internals follow.

INTERVIEW QUESTIONS

Q. Are type hints enforced at runtime?
A. No — they're for humans and static checkers (mypy/ty); they catch bugs early and power tooling but don't affect execution.

Q. What does `Optional[T]` mean?
A. The value is `T` or `None` (same as `T | None`); a checker then forces you to handle the None case.

Q. What is a `Protocol`?
A. Structural typing — any object with the required methods satisfies it, no inheritance needed (typed duck typing).

22. Structural Pattern Matching

Since Python 3.10, `match / case` matches the **shape** of data, not just equality. It destructures sequences, mappings, and objects, and binds names as it goes — far clearer than a deep `if/elif` chain.

```
def handle(event: dict):
    match event:
        case {"type": "click", "x": x, "y": y}:
            return f"click at {x},{y}"          # binds x, y
        case {"type": "key", "code": code}:
            return f"key {code}"
        case {"type": t}:
            return f"unknown event {t}"
        case _:
            return "not an event"               # wildcard / default

# Sequence patterns, guards, and OR-patterns:
match point:
    case (0, 0):              return "origin"
    case (x, 0) | (0, x):     return f"on an axis at {x}"
    case (x, y) if x == y:    return "on the diagonal"     # guard
    case (x, y):              return f"({x}, {y})"
```

ℹ️ **NOTE** A bare name in a `case` *captures* (binds anything); to match against an existing constant, use a dotted name (e.g. an `Enum` member like `Color.RED`) or a literal. This trips people up first time.

❖ **IN PRODUCTION** Pattern matching shines for parsing structured inputs: webhook/event payloads, ASTs, protocol messages, command dispatch. It pairs naturally with dataclasses — `case Point(x=x, y=y):` matches and destructures an object in one line.

INTERVIEW QUESTIONS

Q. What does `match/case` add over if/elif?

A. It matches the structure of data (sequences, mappings, objects), destructures, and binds names in one construct — clearer for parsing structured inputs.

Q. A gotcha with a bare name in a `case`?

A. A bare name captures (binds anything); to match an existing constant, use a dotted name (e.g. an Enum member) or a literal.

23. Modules and __name__

Every module has a built-in `__name__`. When you run a file directly, Python sets its `__name__` to `"__main__"`; when the file is imported, `__name__` is the module's name instead. That single fact powers the familiar guard.

```
# tool.py
def main():
    ...

if __name__ == "__main__":   # True only when run as 'python tool.py'
    main()                   # skipped when 'import tool' elsewhere
```

- **Run directly** python tool.py → __name__ == "__main__" → main() runs.
- **Imported** import tool → __name__ == "tool" → main() does not run, but its functions are available.

❖ **IN PRODUCTION** The guard lets one file be *both* an importable library and a runnable script/CLI. It is also *required* for multiprocessing on spawn platforms (Chapter 6) — without it, child processes re-run your top-level code.

INTERVIEW QUESTIONS

Q. What is `__name__` and the `__main__` guard for?

A. `__name__` is `'__main__'` when a file is run directly and the module name when imported; the guard runs code only on direct execution, so a file can be both library and script.

Q. Why is the guard required for multiprocessing?

A. On spawn, children re-import the module; without the guard, top-level process-spawning code re-runs in every child.

24. Environments and Packaging: pip, Poetry, uv

Every project should run in an isolated environment with pinned dependencies, so it builds the same way on every machine and in CI. The tooling has three generations.

Tool	What it does	Where it fits
`pip` + `venv`	Install packages; create an isolated env (manual, per project)	The universal baseline; always present
Poetry	Resolves dependencies, manages the venv, locks versions, and publishes — all from one `pyproject.toml`	Library authoring; teams already standardised on it
uv	Single Rust binary that replaces pip + venv + pyenv + pip-tools + pipx; 10–100x faster; universal lockfile	New projects and CI in 2026; the emerging default

Why a resolver matters

Plain `pip` installs packages without globally resolving conflicts first, so two libraries needing different versions of a shared dependency can leave you in a broken state. Poetry and uv **resolve the whole graph** before installing and write a **lockfile** pinning exact versions — reproducible installs everywhere.

✦ **2026 UPDATE** In 2026 `uv` (from Astral, the Ruff team; OpenAI announced an acquisition of Astral in March 2026, with uv staying open source) has become the de-facto standard for new work — roughly 10–100x faster than pip and ~5–10x faster than Poetry, with built-in Python-version management (`uv python install 3.14`) and a universal lockfile. `uv pip ...` is a drop-in pip replacement, so migration is near-zero-risk. Poetry 2.x is still excellent for library *publishing*. For ML stacks with CUDA/system libraries, conda/mamba still handle the non-Python dependencies uv won't.

```
# uv: start a project, add deps, run -- no manual venv activation
uv init myapp && cd myapp
uv add fastapi "uvicorn[standard]"
uv run uvicorn main:app --reload      # creates/maintains the venv for you

# CI is where the speed shows most:
uv sync --frozen                      # install exactly from the lockfile
uv run pytest
```

❖ **IN PRODUCTION** Pair the package manager with `ruff` (lint + format, also from Astral) and a type checker (`mypy` or `ty`) in CI. That trio — fast installs, fast lint/format, static types — is the modern Python project's quality backbone.

INTERVIEW QUESTIONS

Q. Why use a resolver/lockfile instead of plain `pip install`?

A. pip installs without globally resolving conflicts; Poetry/uv resolve the whole graph and write a lockfile for reproducible installs across machines and CI.

Q. Where does uv fit in 2026?

A. It's the fast Rust single-tool replacement for pip+venv+pyenv+pip-tools — the default for new projects and CI; Poetry stays strong for publishing, conda/mamba for system/CUDA deps.

Appendix A — Quick Reference

Mutability at a glance

Immutable	Mutable
`int`, `float`, `bool`, `complex`	`list`
`str`, `bytes`	`dict`
`tuple`, `frozenset`	`set`, `bytearray`

Average-case complexity of common operations

Operation	list	dict / set	deque
Index / key lookup	O(1) / O(n) search	O(1)	O(1) ends
Append / add	O(1) amortised	O(1)	O(1) both ends
Pop from front	O(n)	—	O(1)
Membership `in`	O(n)	O(1)	O(n)

Concurrency cheat-sheet

If your work is…	Use
I/O-bound, tens of tasks	Threading
I/O-bound, thousands of connections	AsyncIO
CPU-bound	Multiprocessing (or free-threaded build, 3.14+)

Version & currency notes (2026)

- Python 3.14 (Oct 2025) makes the free-threaded / no-GIL build officially supported but still opt-in; default-on is a later phase.
- uv is the fastest-growing package manager and the recommended starting point for new projects; Poetry remains strong for publishing; pip is the ever-present baseline.
- Modern syntax preferred in new code: `list[int]` / `dict[str,int]` over `typing.List`; `X | None` over `Optional[X]`; `match/case` for structured dispatch.

— End of cookbook —

PART VIII

Testing

25. Testing with pytest

Tests are how you change code without fear. **pytest** is the de-facto framework: plain `assert` statements with rich failure introspection, plus fixtures, parametrization, and a deep plugin ecosystem — far less ceremony than class-based `unittest`.

Discovery and the plain-assert style

pytest auto-discovers files named `test_*.py` and functions named `test_*`. You assert with the built-in operator; pytest rewrites it to show the actual values on failure.

```
# test_math.py
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5            # on failure pytest prints both sides

def test_add_negatives():
    assert add(-1, -1) == -2

# run:  pytest -q
```

Parametrize instead of copy-pasting

One test, many cases — no duplicated bodies:

```
import pytest

@pytest.mark.parametrize("a, b, expected", [
    (2, 3, 5),
    (-1, 1, 0),
    (0, 0, 0),
])
def test_add(a, b, expected):
    assert add(a, b) == expected     # runs 3 separate cases
```

Asserting on errors

```
import pytest

def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        1 / 0

def test_bad_input():
    with pytest.raises(ValueError, match="positive"):
        validate(-1)                 # also checks the message
```

- **Markers** — tag tests (@pytest.mark.slow) to select or skip subsets: pytest -m 'not slow'.

- **Structure** — keep tests beside the package or in a tests/ folder; one behaviour per test, named for what it asserts.

❖ **IN PRODUCTION** Wire pytest into CI with coverage (`pytest --cov`) and fail the build under a threshold. The test pyramid still holds: many fast unit tests, fewer integration tests, a handful of end-to-end. A green suite is what lets you (and AI tools) refactor aggressively.

INTERVIEW QUESTIONS

Q. Why pytest over unittest?
A. Plain `assert` with rich introspection, fixtures, parametrization, and a big plugin ecosystem — far less boilerplate than class-based unittest.

Q. How do you test many inputs without duplicating tests?
A. `@pytest.mark.parametrize` feeds multiple input/expected pairs into one test function.

Q. How do you assert that code raises?
A. `with pytest.raises(ValueError):` around the call, optionally checking the message with `match=`.

26. Fixtures, Mocking and Test Doubles

Fixtures: reusable setup and teardown

A **fixture** provides a prepared object to any test that names it as a parameter. `yield` splits setup from teardown, and *scope* controls how often it runs.

```
import pytest

@pytest.fixture
def db():
    conn = connect(":memory:")       # setup
    yield conn                       # value handed to the test
    conn.close()                     # teardown, always runs

def test_insert(db):                 # 'db' injected by name
    db.execute("INSERT ...")
    assert db.count() == 1

@pytest.fixture(scope="session")     # created once for the whole run
def app():
    return build_app()
```

Shared fixtures live in `conftest.py`, auto-discovered by every test in that directory tree — no imports needed.

Mocking: replace the boundary, not the logic

Mock the slow, external, or non-deterministic dependencies — network calls, the clock, third-party APIs — so tests stay fast and deterministic. The golden rule: **patch where the name is looked up**, not where it is defined.

```
from unittest.mock import patch

# app/weather.py imports requests and calls requests.get(...)
def test_forecast():
    with patch("app.weather.requests.get") as mock_get:   # patch in app.weather
        mock_get.return_value.json.return_value = {"temp": 21}
        assert get_temp("Dhaka") == 21
        mock_get.assert_called_once()        # a mock also verifies the call
```

```
# pytest's monkeypatch does the same, fixture-style:
def test_env(monkeypatch):
    monkeypatch.setenv("API_KEY", "test")
```

Double	Behaviour	Use when
Stub	Returns canned values	You just need a dependency to 'be there'
Mock	Canned values + records/asserts calls	You must verify an interaction happened
Fake	Working lightweight implementation	An in-memory DB / queue is cleaner than mocking

⚠️ **GOTCHA** Over-mocking couples tests to implementation detail — they pass while the system is broken. Mock external boundaries; let internal logic run for real. If patching needs deep paths, that often signals a design that wants a seam (dependency injection).

INTERVIEW QUESTIONS

Q. What is a fixture and why use it?

A. A reusable setup/teardown injected into tests by name; fixtures have scopes (function/module/session) and live in `conftest.py` for sharing.

Q. When do you mock, and the rule of thumb?

A. Mock at the boundary — external, slow, or non-deterministic dependencies (network, time, APIs) — and patch where the name is used, not where it's defined.

Q. Mock vs stub vs fake?

A. A stub returns canned values; a mock also asserts how it was called; a fake is a working lightweight implementation (e.g. an in-memory DB).

PART IX

Data Structures and Algorithms in Python

27. Key Data Structures

Interview and systems work both reward picking the right structure. Python gives you most of them in the standard library — you rarely implement them from scratch, but you must know the trade-offs.

Stack, queue, priority queue

```
# Stack: a plain list (LIFO)
stack = []
stack.append(1); stack.append(2)
stack.pop()                          # 2

# Queue (FIFO): collections.deque -- O(1) at both ends
from collections import deque
q = deque()
q.append(1); q.append(2)
q.popleft()                          # 1   (list.pop(0) would be O(n))

# Priority queue: heapq (binary min-heap over a list)
import heapq
h = []
heapq.heappush(h, (2, "b"))
heapq.heappush(h, (1, "a"))
heapq.heappop(h)                     # (1, "a")  -- smallest first; push/pop O(log n)
```

Trees, graphs, tries

```
# Binary tree node
class Node:
    def __init__(self, val):
        self.val = val
        self.left = self.right = None

# Graph: dict of adjacency lists
from collections import defaultdict
graph = defaultdict(list)
graph["A"].append("B")               # edge A -> B
graph["A"].append("C")

# Trie (prefix tree) via nested dicts
trie = {}
def insert(word):
    node = trie
    for ch in word:
        node = node.setdefault(ch, {})
    node["$"] = True                 # end-of-word marker
```

Structure	Python	Lookup / key op	Strength
Hash map	`dict`	O(1) avg	General-purpose association
Hash set	`set`	O(1) avg	Membership, dedup
Stack / queue	`list` / `deque`	O(1) ends	LIFO / FIFO
Heap	`heapq`	O(log n) push/pop	Top-k, schedulers, Dijkstra
Ordered map	`dict` (insertion order)	O(1) avg	Dicts keep insertion order (3.7+)

❖ **IN PRODUCTION** Most "which data structure?" questions reduce to: need order → list/deque; need fast membership/dedup → set; need association → dict; need the smallest/largest repeatedly → heap. Reaching for the wrong one (e.g. `list` membership in a hot loop, O(n)) is a classic scaling bug.

INTERVIEW QUESTIONS

Q. Implement a stack, a queue, and a priority queue in Python?

A. Stack: a list (`append`/`pop`). Queue: `collections.deque` (`append`/`popleft`). Priority queue: `heapq` over a list (a binary min-heap).

Q. Is there a built-in heap, and what's its complexity?

A. `heapq` gives a min-heap over a list; push and pop are O(log n), peek (`heap[0]`) is O(1).

Q. How do you represent a graph?

A. Usually a dict of adjacency lists (`defaultdict(list)`); for weights, store (neighbour, weight) tuples or a nested dict.

28. Key Algorithms

A small set of patterns covers most algorithmic problems. Python's standard library implements several so you don't hand-roll them.

Binary search with bisect

```
import bisect
xs = [1, 3, 4, 7, 9]
bisect.bisect_left(xs, 4)          # 2  -> index of 4  (O(log n))
bisect.insort(xs, 5)               # keep list sorted on insert
```

Two pointers and sliding window

```
# Two pointers: is the (sorted) array a pair summing to target?
def has_pair(xs, target):
    i, j = 0, len(xs) - 1
    while i < j:
        s = xs[i] + xs[j]
        if s == target: return True
        if s < target:  i += 1
        else:           j -= 1
    return False

# Sliding window: longest run of distinct chars
def longest_unique(s):
    seen, start, best = {}, 0, 0
    for end, ch in enumerate(s):
        if ch in seen and seen[ch] >= start:
            start = seen[ch] + 1
```

```
            seen[ch] = end
            best = max(best, end - start + 1)
        return best
```

Graph traversal: BFS and DFS

```
from collections import deque

def bfs(graph, start):
    seen, q = {start}, deque([start])
    while q:
        node = q.popleft()
        for nxt in graph[node]:
            if nxt not in seen:
                seen.add(nxt); q.append(nxt)
    return seen                        # O(V + E)

def dfs(graph, node, seen=None):
    seen = seen or set()
    seen.add(node)
    for nxt in graph[node]:
        if nxt not in seen:
            dfs(graph, nxt, seen)
    return seen
```

Recursion + memoization (dynamic programming)

```
from functools import lru_cache

@lru_cache(maxsize=None)             # turns O(2^n) into O(n)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)
```

❖ **IN PRODUCTION** Recognise the pattern before coding: sorted input → binary search / two pointers; contiguous subarray → sliding window; shortest path on unweighted graph → BFS; overlapping subproblems → memoisation. Naming the pattern out loud is exactly what interviewers listen for, and it ties back to `Timsort` and `heapq` from earlier chapters.

INTERVIEW QUESTIONS

Q. How do you binary-search without writing it?

A. `bisect` — `bisect_left` and `insort` over a sorted list give O(log n) search and ordered insertion.

Q. BFS vs DFS in Python?

A. BFS uses a `deque` queue; DFS uses recursion or an explicit stack. Track visited to avoid cycles; both are O(V+E).

Q. How do you memoize a recursive algorithm?

A. `@functools.lru_cache` (or `@cache`) turns exponential recursion (e.g. naive Fibonacci) into linear time.

PART X

Python in Production

29. Debugging

Debugging is a method, not luck: **reproduce → isolate → hypothesise → verify → fix → add a regression test**. The senior signal is changing one variable at a time and observing, rather than editing hopefully.

Interactive debugging with pdb

Drop `breakpoint()` anywhere to enter the debugger (PEP 553). Step with `n` (next), `s` (step into), inspect with `p expr`, continue with `c`.

```
def process(order):
    total = compute(order)
    breakpoint()                    # drops into pdb here
    return apply_discount(total)

# After an uncaught exception, post-mortem the crash frame:
import pdb; pdb.pm()
```

Read the traceback bottom-up

The last line is the exception and message; the frames above are the call chain. Read bottom-to-top: the deepest frame is where it blew up, the frames above show how you got there.

Profiling: measure before optimising

```
import cProfile
cProfile.run("expensive()")         # call-level hotspots: ncalls, cumtime

# microbenchmark a snippet:
from timeit import timeit
timeit("sorted(data)", globals=globals(), number=1000)
```

Memory: tracemalloc

```
import tracemalloc
tracemalloc.start()
# ... run the workload ...
snap = tracemalloc.take_snapshot()
for stat in snap.statistics("lineno")[:10]:
    print(stat)                     # top allocations by source line
```

❖ **IN PRODUCTION** Profile *before* you optimise — intuition about hotspots is usually wrong. In production, reach for logs, metrics, and distributed tracing (next chapter) before a local debugger; you often can't attach pdb to a live service, but a good trace tells you where the time and errors went.

✦ **2026 UPDATE** AI assistants are strong rubber-duck debuggers — paste the traceback and a minimal repro. But verify the fix: AI confidently proposes plausible-but-wrong causes, so confirm with a failing test that turns green.

INTERVIEW QUESTIONS

Q. How do you debug interactively?

A. Drop a `breakpoint()` to enter pdb; step with n/s, inspect with p, continue with c. For crashes, post-mortem with `pdb.pm()`.

Q. How do you find a performance bottleneck?

A. Profile first — `cProfile` for call-level hotspots, `timeit` for microbenchmarks — then optimise what the data shows, not what you guess.

Q. How do you track high memory or a leak?

A. `tracemalloc` to snapshot and diff allocations by line; `sys.getsizeof` for object sizes; watch for unbounded caches and lingering references.

30. Logging, Monitoring and OpenTelemetry

Use logging, not print

The `logging` module gives you **levels** (DEBUG/INFO/WARNING/ERROR/CRITICAL), **handlers** (where output goes), and **formatters** — all configurable centrally, without touching call sites. `print` has none of that and pollutes stdout.

```
import logging
logger = logging.getLogger(__name__)     # module-scoped logger, never the root

def charge(order):
    logger.info("charging order %s for %s", order.id, order.amount)
    try:
        gateway.charge(order)
    except GatewayError:
        logger.exception("charge failed for %s", order.id)   # includes traceback
```

Structured logging

Emit machine-parseable key/value records (usually JSON) instead of free text, so logs are queryable and aggregatable in tools like ELK, Loki, or a cloud logging backend.

```
# with structlog (or a JSON formatter on stdlib logging)
import structlog
log = structlog.get_logger()
log.info("payment", order_id=order.id, amount=order.amount, currency="USD")
# -> {"event": "payment", "order_id": 42, "amount": 19.9, "currency": "USD"}
```

The three pillars: logs, metrics, traces

- **Logs** — discrete events ('charge failed for order 42').
- **Metrics** — aggregated numbers over time (request rate, p95 latency, error count); counters, gauges, histograms.
- **Traces** — the path of one request across services, with timing per span — how you find where latency actually goes.

OpenTelemetry: instrument once, export anywhere

OpenTelemetry (OTel) is the vendor-neutral CNCF standard and SDK for all three pillars. You instrument your code against the OTel API; an exporter ships the data to whatever backend you choose — Jaeger, Prometheus, Datadog, Grafana — so you are never locked to one vendor. Auto-instrumentation covers popular libraries (FastAPI, Django, requests, psycopg) with little or no code change.

```
from opentelemetry import trace
tracer = trace.get_tracer(__name__)
```

```python
def handle_request(req):
    with tracer.start_as_current_span("handle_request") as span:
        span.set_attribute("user.id", req.user_id)
        data = fetch_from_db(req)          # nested spans show DB time
        return render(data)

# Auto-instrument a whole app with no code changes:
#   opentelemetry-instrument --traces_exporter otlp python app.py
```

✦ **2026 UPDATE** OpenTelemetry is now the industry default for observability — it merged the older OpenTracing and OpenCensus projects and is the second-most-active CNCF project after Kubernetes. Standardising on OTel (rather than a vendor SDK) is the modern, portable choice and a strong thing to say in a systems interview.

❖ **IN PRODUCTION** Set logger level by environment (DEBUG locally, INFO in prod), attach a `trace_id` to every log line so logs and traces correlate, and alert on metrics (error rate, p95) rather than on individual log lines.

INTERVIEW QUESTIONS

Q. Why use `logging` instead of `print`?

A. Levels, handlers, formatting, and routing (files, stdout, aggregators) configurable without touching call sites; `print` has none of that and pollutes stdout.

Q. What is structured logging and why does it matter?

A. Emitting machine-parseable key/value records (often JSON) instead of free text, so logs are queryable and aggregatable in tools like ELK or Loki.

Q. What is OpenTelemetry and why standardise on it?

A. A vendor-neutral CNCF standard and SDK for traces, metrics, and logs with auto-instrumentation; instrument once and export to any backend (Jaeger, Prometheus, Datadog) instead of coupling code to one vendor.

31. Caching

Caching trades memory (or staleness) for speed by remembering results you'd otherwise recompute or refetch. Python gives you in-process caching for free; external caches (Redis) scale it across processes and machines.

In-process memoisation

```python
from functools import lru_cache, cache

@lru_cache(maxsize=1024)            # bounded cache, evicts least-recently-used
def geocode(address):
    return expensive_lookup(address)

@cache                              # unbounded (3.9+); use only for small key spaces
def factorial(n):
    return 1 if n <= 1 else n * factorial(n - 1)
```

Use these only for *pure* functions (same inputs → same output) with hashable arguments. For time-based expiry, `cachetools` offers `TTLCache`.

Cache-aside with an external store

```python
import json, redis
r = redis.Redis()
```

```python
def get_user(uid):
    key = f"user:{uid}"
    cached = r.get(key)
    if cached:                          # cache hit
        return json.loads(cached)
    user = db.fetch_user(uid)           # miss -> load from source
    r.setex(key, 300, json.dumps(user))   # populate with a 300s TTL
    return user
```

⚠ **GOTCHA** "There are only two hard things in computer science: cache invalidation and naming." Stale data, *thundering herd* (many clients recomputing the instant a hot key expires), and caching things that must be fresh are the real traps. Pick TTLs deliberately, and never let a cache silently stand in for a read that must be authoritative (balances, permissions).

INTERVIEW QUESTIONS

Q. How do you add memoization in one line?

A. `@functools.lru_cache(maxsize=...)` (or `@cache`) on a pure function caches results by arguments.

Q. What is the cache-aside pattern?

A. Check the cache; on a miss, load from the source, populate the cache, then return — the most common app-level pattern (e.g. with Redis).

Q. What are the hard parts of caching?

A. Invalidation and staleness: choosing TTLs, avoiding thundering-herd on expiry, and deciding when a cache may not stand in for a fresh read.

32. Queues and Task Processing

A queue decouples producers from consumers: hand off work to be done later, by someone else, at their pace. In Python this spans in-process queues up to distributed task systems.

In-process queues

```python
# Thread-safe producer/consumer with queue.Queue
import queue, threading
q = queue.Queue()

def worker():
    while True:
        item = q.get()              # blocks until an item is available
        process(item)
        q.task_done()

threading.Thread(target=worker, daemon=True).start()
q.put("job-1")
q.join()                            # wait until all items processed
```

- **queue.Queue** — thread-safe, for threads. asyncio.Queue for coroutines; multiprocessing.Queue across processes.

Distributed task queues and brokers

When work must outlive the request, run on separate workers, retry on failure, or scale independently, move to a **task queue** (Celery, RQ, Dramatiq) backed by a broker (Redis, RabbitMQ) — or a log/stream like Kafka for high-throughput event pipelines.

```python
# Celery: define a task, call it asynchronously
from celery import Celery
app = Celery("tasks", broker="redis://localhost:6379/0")

@app.task(bind=True, max_retries=3)
def send_email(self, to):
    try:
        smtp.send(to)
    except TransientError as e:
        raise self.retry(exc=e, countdown=5)   # backoff and retry

send_email.delay("user@example.com")           # returns immediately
```

⚠️ **GOTCHA** Brokers typically guarantee **at-least-once** delivery, so a task can run twice — make consumers **idempotent** (safe to repeat). Also plan for ordering, retry/backoff, and a dead-letter queue for messages that keep failing.

INTERVIEW QUESTIONS

Q. What in-process queues does Python offer?

A. `queue.Queue` (thread-safe, for threads), `asyncio.Queue` (coroutines), and `multiprocessing.Queue` (across processes).

Q. When reach for Celery/RQ or a broker (Kafka/RabbitMQ)?

A. When work must outlive the request, run on separate workers, retry, or scale independently — background task processing and service decoupling.

Q. What delivery issues must you plan for?

A. At-least-once delivery means consumers must be idempotent; also handle ordering, retries/backoff, and dead-letter queues.

33. Security Essentials

Most Python security incidents come from a short list of language-level footguns. Knowing them is table stakes for backend work.

Never deserialise or evaluate untrusted input

```python
import pickle, yaml, json

pickle.loads(untrusted)        # DANGER: can execute arbitrary code
eval(user_input)               # DANGER: arbitrary code execution
yaml.load(text)                # DANGER on old PyYAML -> use safe_load

# Safe alternatives:
json.loads(untrusted)          # data only, no code
yaml.safe_load(text)
```

Parameterise SQL — never format it

```python
# WRONG: string interpolation -> SQL injection
cur.execute(f"SELECT * FROM users WHERE name = '{name}'")

# RIGHT: bound parameters, the driver escapes them
cur.execute("SELECT * FROM users WHERE name = %s", (name,))
```

Secrets and dependencies

- **Secrets** — never hard-code keys/passwords; read from environment or a secret manager (Vault, cloud KMS). Keep them out of source control and logs.
- **Dependencies** — pin versions and scan for known CVEs (pip-audit); a vulnerable transitive dependency is the most common real-world breach vector.
- **Input validation** — validate and type-coerce at the boundary (pydantic, schemas); treat all external input as hostile until proven otherwise.

❖ **IN PRODUCTION** Add `pip-audit` (or equivalent) and a static analyser like `bandit` to CI so injection, weak crypto, and vulnerable dependencies fail the build before they ship. Security that isn't automated doesn't happen.

INTERVIEW QUESTIONS

Q. Why is `pickle` dangerous?

A. Unpickling untrusted data can execute arbitrary code; never unpickle data you don't fully trust — prefer JSON or a safe schema.

Q. How do you prevent SQL injection in Python?

A. Use parameterised queries / bound parameters so the driver escapes values; never string-format user input into SQL.

Q. Other Python-specific footguns?

A. `eval`/`exec` on input, `yaml.load` without SafeLoader, hard-coded secrets (use env/secret managers), and unpinned or vulnerable dependencies (scan with pip-audit).

34. Memory and the OS from Python

Chapter 3 covered *how* Python frees memory (refcounting + cyclic GC). This chapter is the practical view: measuring memory, cutting it, and how Python sits on top of the operating system.

Measuring memory

```
import sys, tracemalloc

sys.getsizeof([1, 2, 3])            # shallow size of one object, in bytes

tracemalloc.start()
build_big_structure()
top = tracemalloc.take_snapshot().statistics("lineno")[:5]
for stat in top:
    print(stat)                     # which source lines allocated the most
```

Cutting memory

- **__slots__** — on classes you instantiate by the million, this drops the per-instance __dict__ for a large saving (Chapter 20).
- **Generators / streaming** — yield items instead of building giant lists; constant memory instead of O(n) (Chapter 12).
- **Right-sized types** — array / bytes / numpy arrays instead of lists of boxed Python ints when data is numeric and large.

Python and the OS

A Python *process* gets its own virtual address space; *threads* share it (and the GIL). When you `fork` (Chapter 6), children share the parent's pages **copy-on-write**: reads are free, but the first write to a page — including a refcount update — copies it, so even “read-only” shared data can balloon memory in long-lived workers.

- Other OS resources Python wraps: **file descriptors** (close them — use `with` — or you hit the `ulimit` and get 'too many open files'), sockets, and signals.

❖ **IN PRODUCTION** Two production memory smells: a process that grows forever (an unbounded cache or a reference you never release — find it with `tracemalloc` diffs) and pre-fork servers (Gunicorn) where each worker copies large globals on write. Loading big read-only data *before* forking, and avoiding writes to it, keeps copy-on-write sharing intact.

INTERVIEW QUESTIONS

Q. How do you measure object memory and overall allocations?

A. `sys.getsizeof` for one object's shallow size; `tracemalloc` to snapshot and diff allocations by line over time.

Q. How does `fork` interact with memory?

A. Children share parent pages copy-on-write, so reads are cheap but writes (including refcount updates) trigger copies — why long-lived workers balloon and why fork is unsafe with threads.

Q. Two simple ways to cut Python memory use?

A. `__slots__` on high-count classes (drops per-instance dicts) and generators/streaming instead of building large lists.

A note on scope (what this edition leaves out)

Two topics from the original wish-list were deliberately kept out of this *language* cookbook because they belong to systems/architecture rather than Python itself, and folding them in would break the book's flow:

- **Frameworks & layers** (Django/FastAPI architecture, clean/hexagonal layering) — covered better in the backend interview-readiness materials, where request lifecycle and separation of concerns sit alongside system design.
- **TCP/IP foundations** (the OSI model, TCP vs UDP, TLS handshake) — general networking, not Python; it lives with the backend networking track.

Concurrency and parallelism are not missing — they are Part II (Chapters 4–8), covered in depth. If a later edition adds the two topics above, they would form a separate “Python for Web Backends” part rather than disturbing the chapters here.